

Splay Tree Construction

1 | Algorithms for Splay Tree Construction

To construct a Splay Tree, we perform three major operations:

- 1. Splaying:** Moves the accessed node to the root using rotations.
- 2. Insertion:** Inserts a new node and then splays it to the root.
- 3. Search:** Searches for a node and then splays it to the root.

1.1 | Splay Operation

Algorithm 1 Splay Operation

```

1: function SPLAY( $T, X$ )                                ▷  $T$  is the root,  $X$  is the key to splay
2:   if  $X$  is root or  $X$  is NULL then
3:     return  $T$ 
4:   end if
5:   if  $X$  is left child of parent  $P$  then
6:     if  $P$  is root then                                     ▷ Zig Rotation
7:       Perform Right Rotation on  $P$ 
8:     else if  $X$  is left child of grandparent  $G$  then       ▷ Zig-Zig Rotation
9:       Perform Right Rotation on  $G$ 
10:      Perform Right Rotation on  $P$ 
11:     else if  $X$  is right child of grandparent  $G$  then     ▷ Zig-Zag Rotation
12:       Perform Left Rotation on  $P$ 
13:       Perform Right Rotation on  $G$ 
14:     end if
15:   else if  $X$  is right child of parent  $P$  then
16:     if  $P$  is root then                                     ▷ Zig Rotation
17:       Perform Left Rotation on  $P$ 
18:     else if  $X$  is right child of grandparent  $G$  then     ▷ Zig-Zig Rotation
19:       Perform Left Rotation on  $G$ 
20:       Perform Left Rotation on  $P$ 
21:     else if  $X$  is left child of grandparent  $G$  then     ▷ Zig-Zag Rotation
22:       Perform Right Rotation on  $P$ 
23:       Perform Left Rotation on  $G$ 
24:     end if
25:   end if
26:   return  $X$                                               ▷  $X$  is now at the root
27: end function

```

1.2 | Insertion in Splay Tree

Algorithm 2 Insert Operation

```

1: function INSERT( $T, X$ )                                ▷  $T$  is the root,  $X$  is the key to insert
2:   if  $T$  is NULL then
3:     Create a new node with key  $X$ 
4:     return  $X$  as root
5:   end if
6:   Perform BST Insertion of  $X$ 
7:   Splay( $T, X$ )
8:   return  $X$  as new root
9: end function

```

1.3 | Search Operation

Algorithm 3 Search Operation

```

1: function SEARCH( $T, X$ )
2:   Perform standard BST Search for  $X$ 
3:   if  $X$  is found then
4:     Splay( $T, X$ )
5:   else
6:     Splay( $T$ , last visited node)
7:   end if
8:   return root of updated tree
9: end function

```

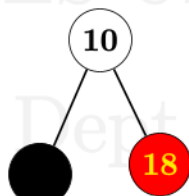
▷ T is the root, X is the key to search

2 | Illustration : 10,18,7,15,16,30,25,40,2,70

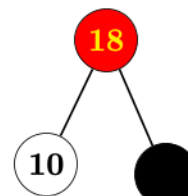
Insert(10)



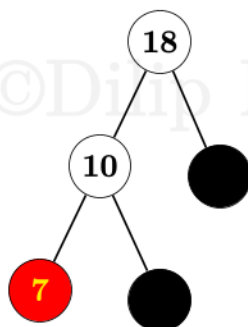
Insert(18)



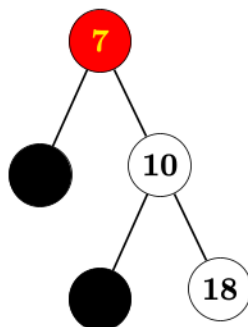
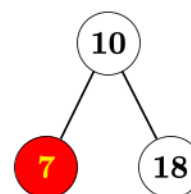
Splay(18)(Zag)

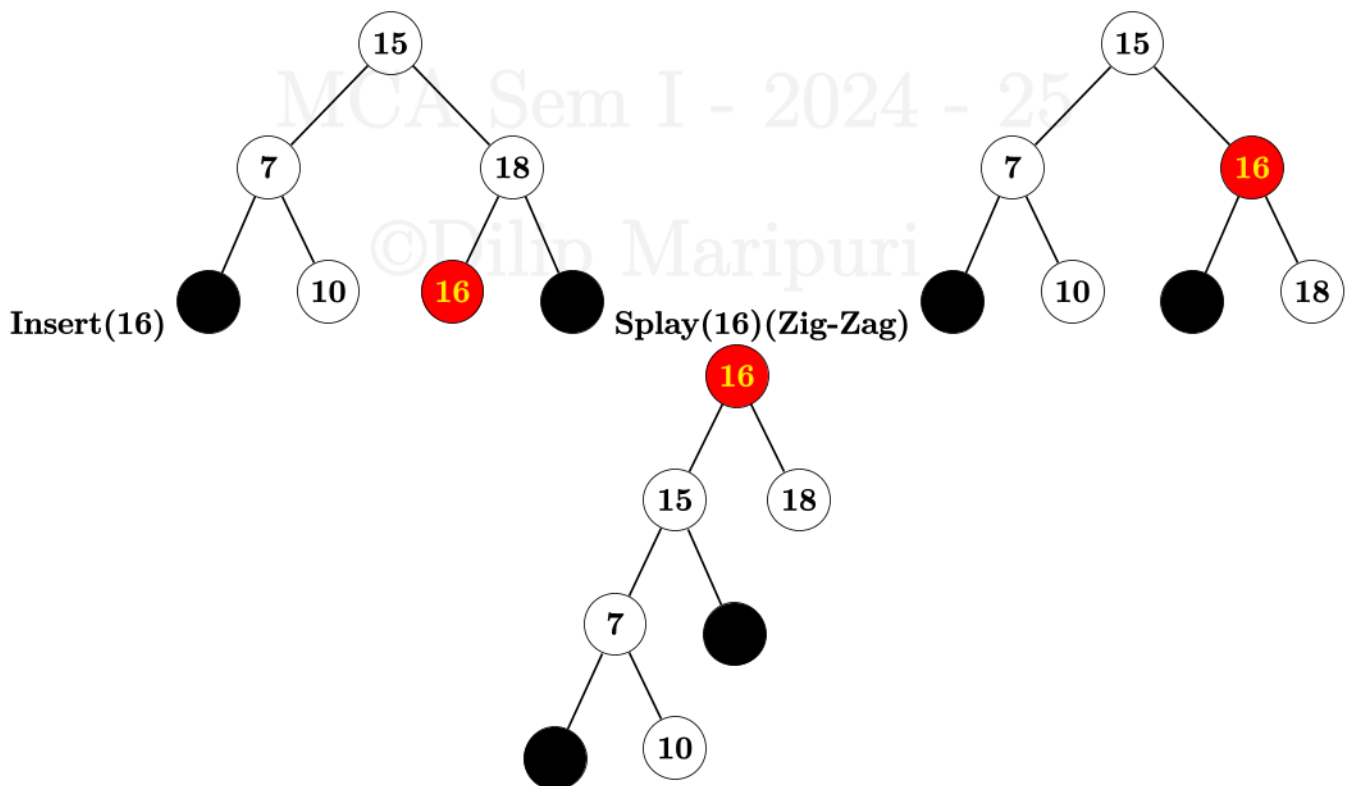
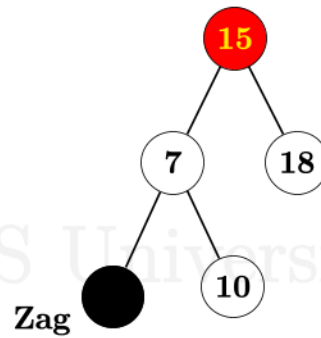
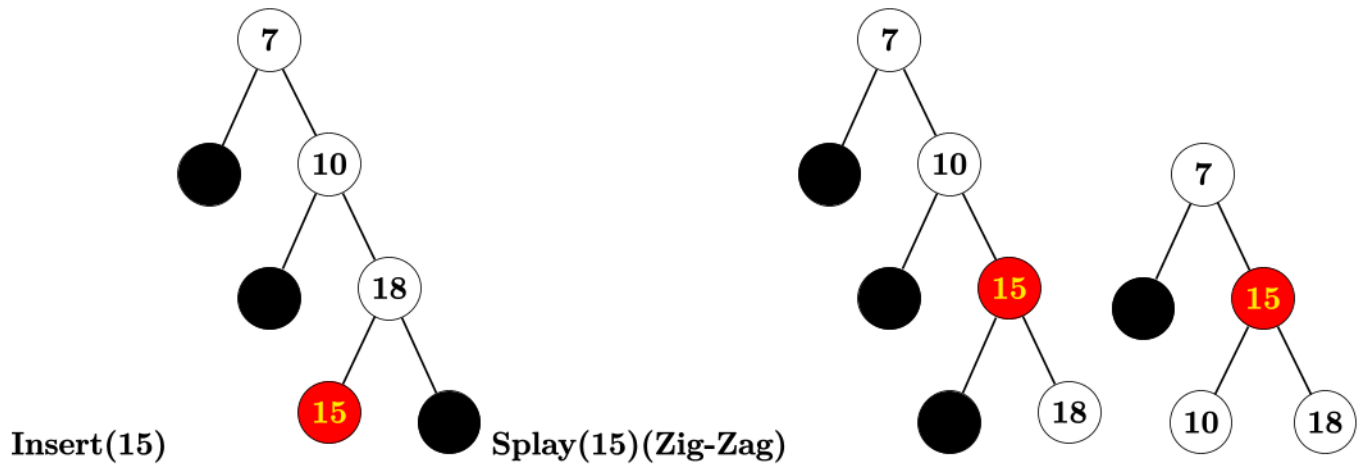


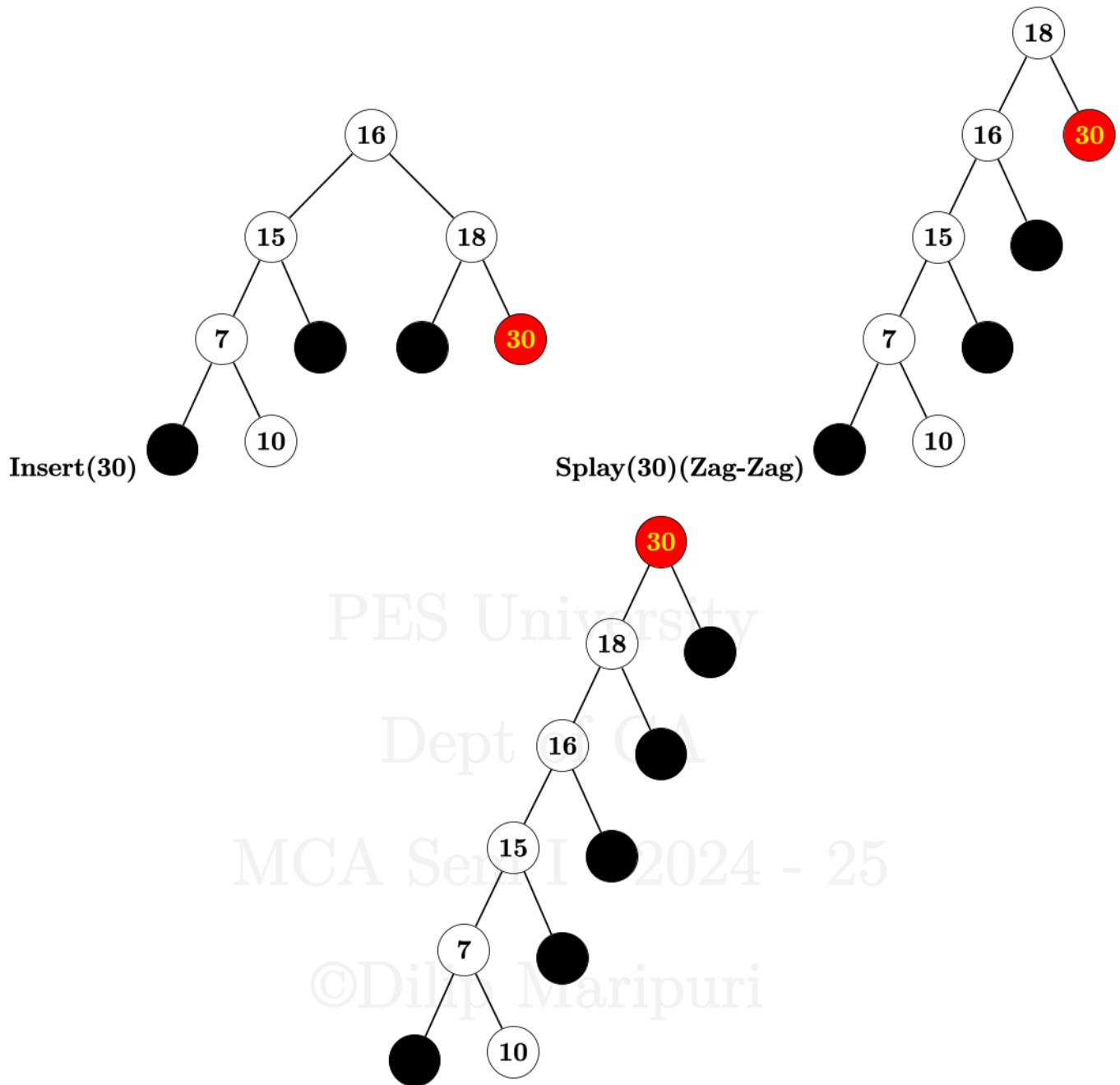
Insert(7)

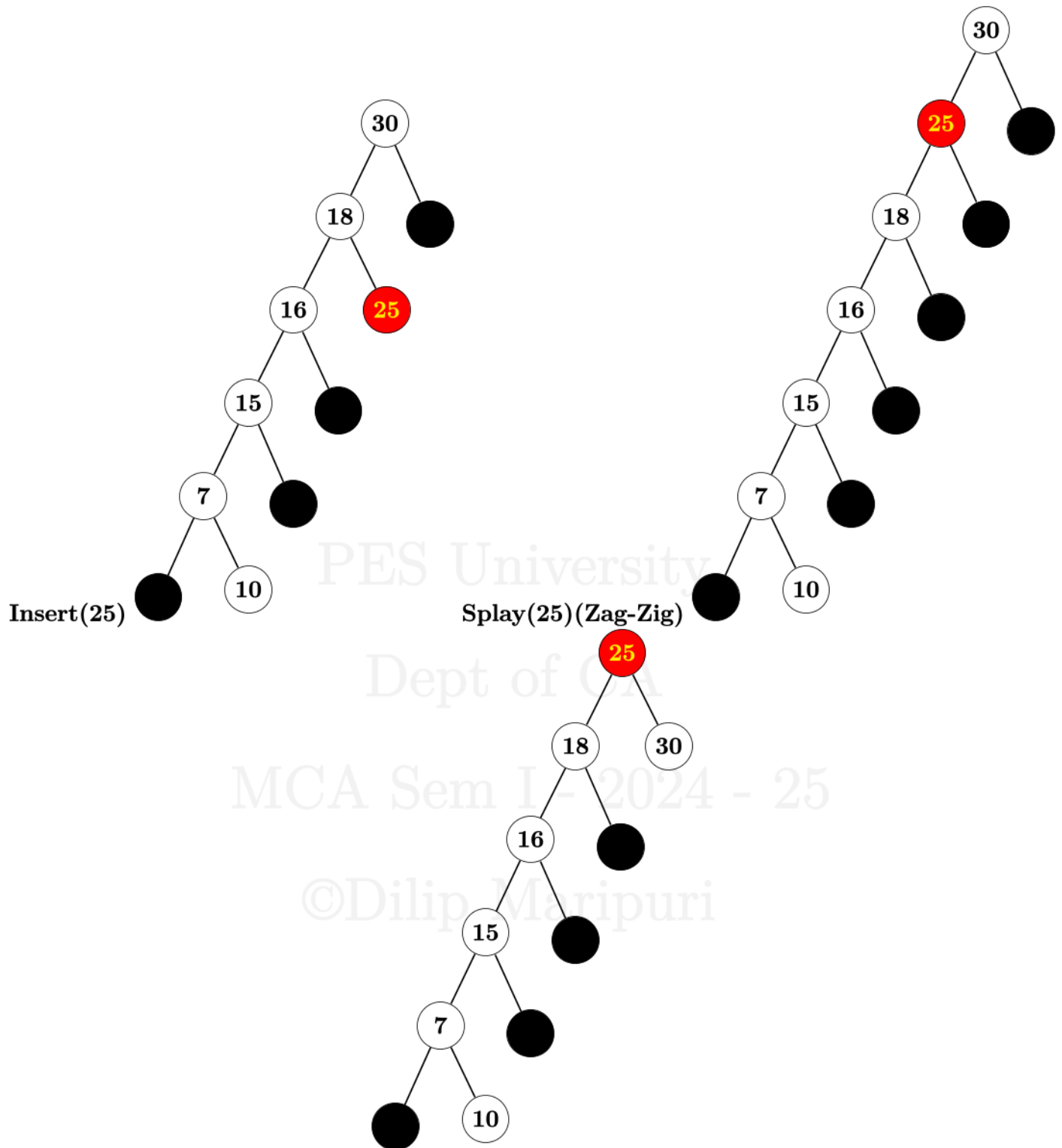


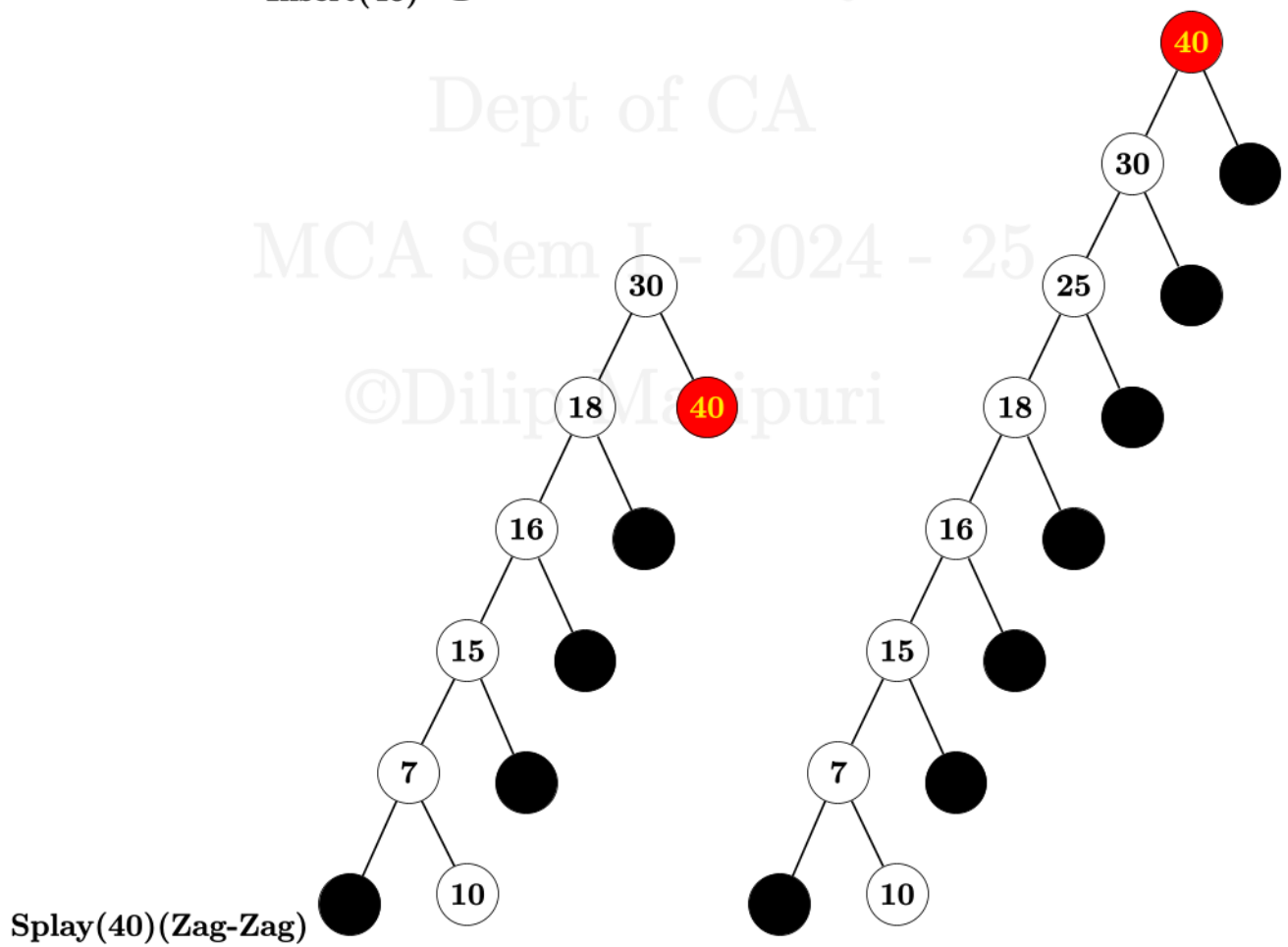
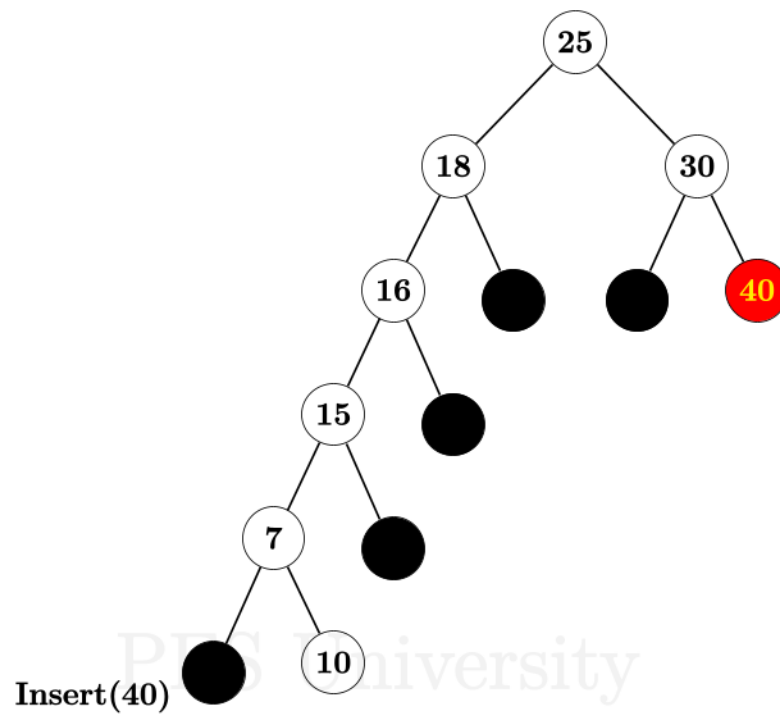
Splay(7)(Zig-Zig)

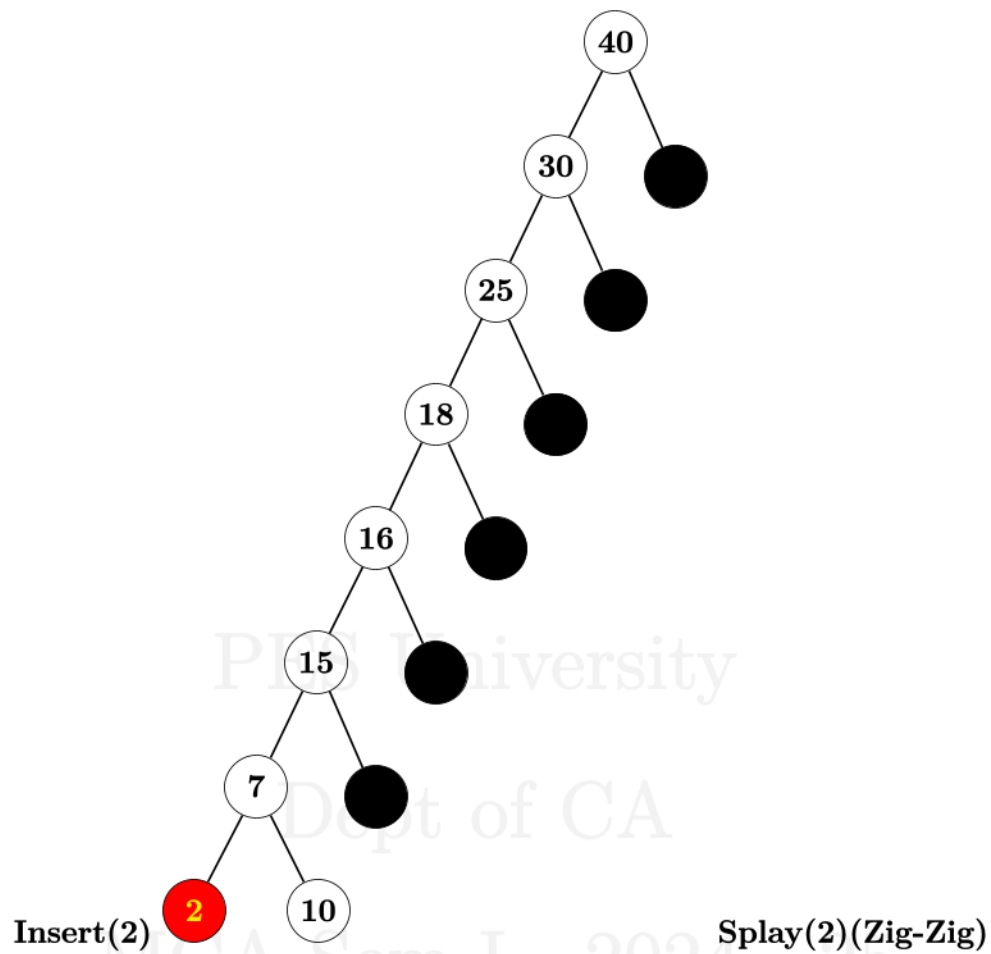


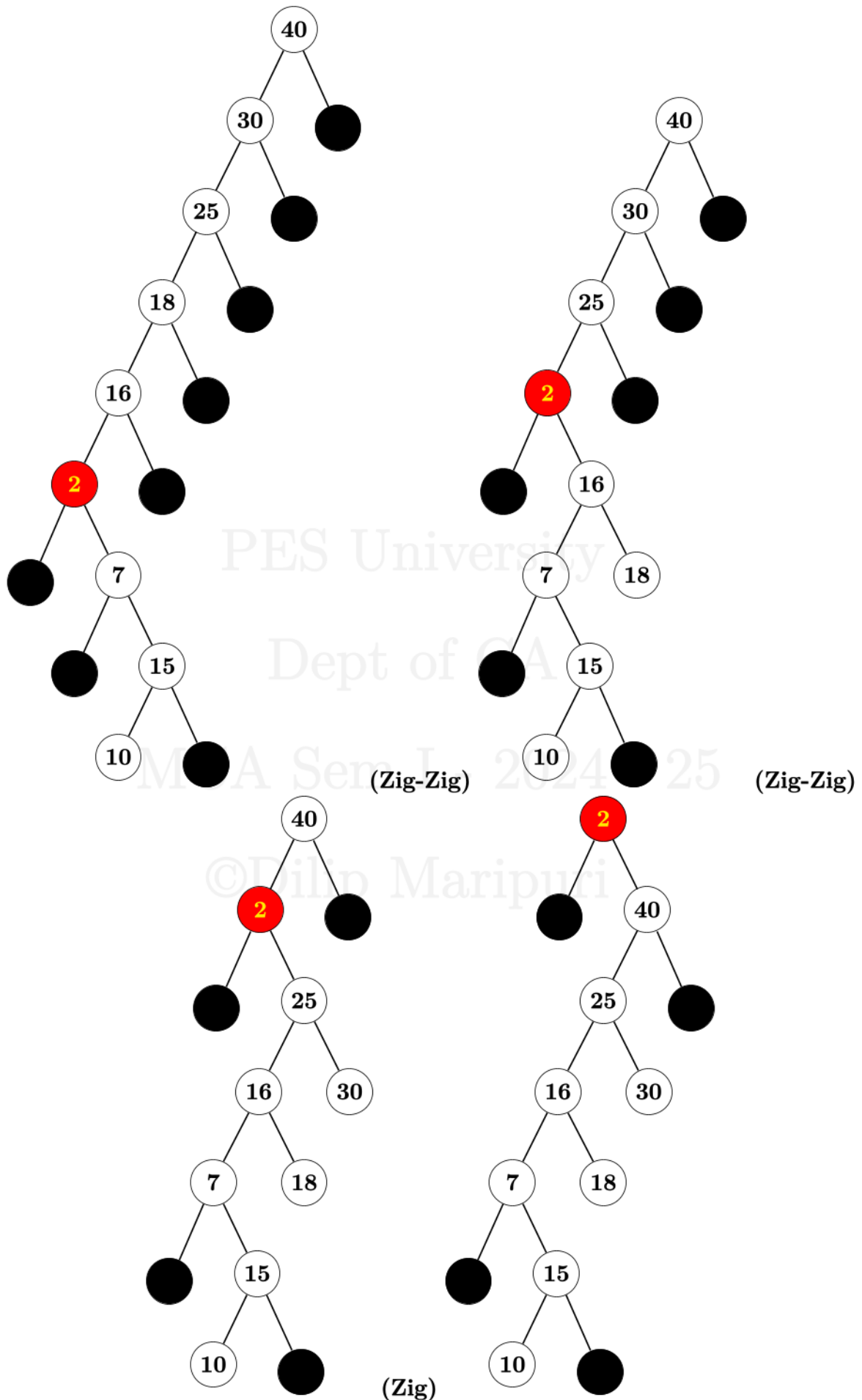


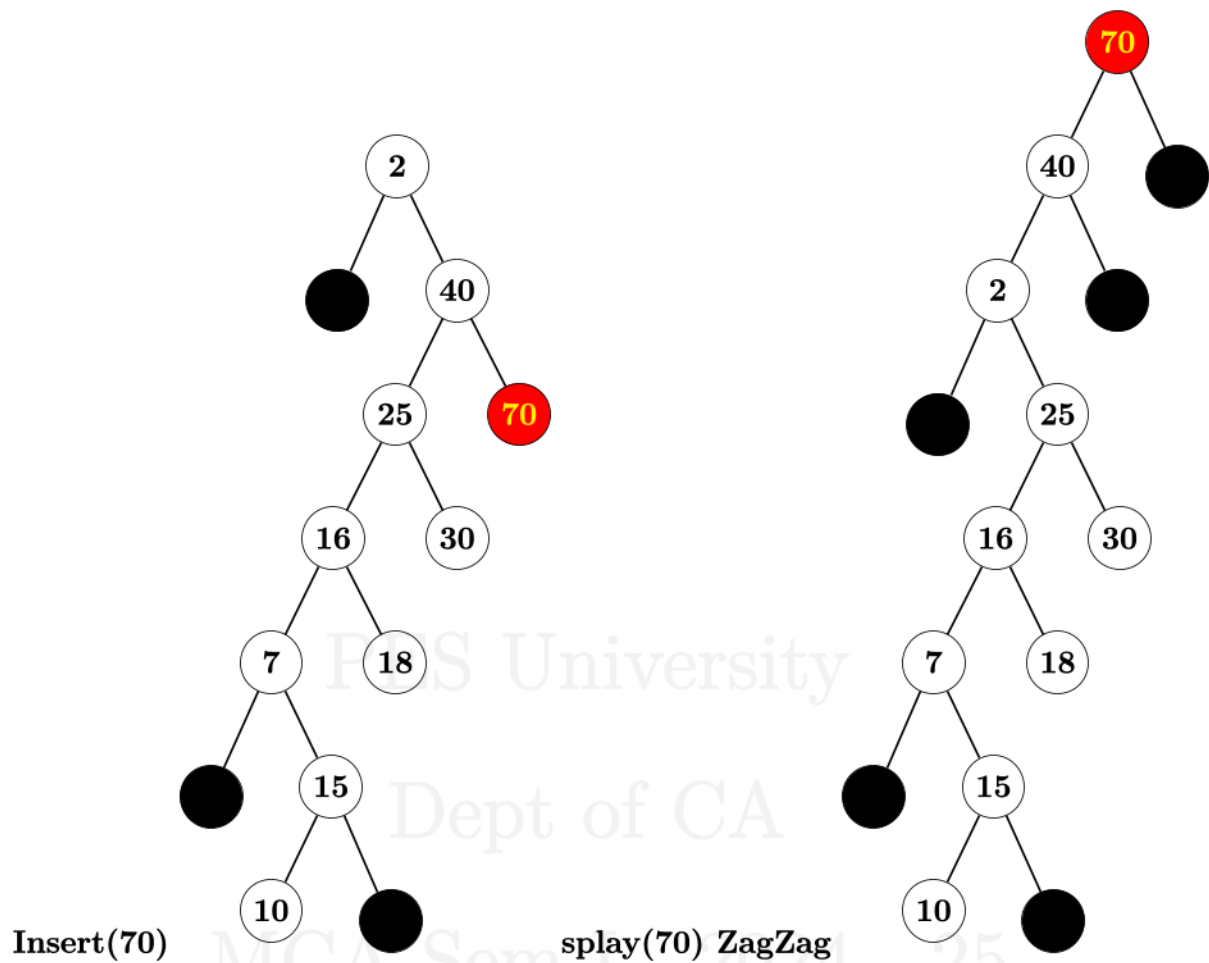












At the end of Splay Tree Construction, The student should be able to tabulate the following data

Element	Zig	Zag	ZigZig	ZagZag	ZigZag	ZagZig
10						
18		1				
7			1			
15		1			1	
16					1	
30				1		
25						1
40				1		
02	1			3		
70				1		
Total	2	1	1	6	3	1

1. How many transformations are performed to splay 16 after it is inserted? **1-ZigZag**
2. Total No.of transformations that are performed to splay 70 after it is inserted : **14**
3. Is there any transformation that is not applied in the process of construction all the elements into the splay tree in the said order? **NIL**